

プログラミングを行う際、後で処理するためにデータを構造化してまとめることができると便利です。Kotlin では、まさにこの目的のためにコレクションが用意されています。

Kotlin has the following collections for grouping items:

コレクションの種類	説明
Lists	項目の順序付き集合
Sets	一意で順序のない項目の集合
Maps	キーが一意であり、1つの値のみにマッピングされるキーと値のペアの集合

各コレクション型は、可変型または読み取り専用型にすることができます。

List

リスト

リストは、項目が追加された順に保存され、重複する項目も許可されます。

読み取り専用のリスト (`List`) を作成するには、`listOf()` 関数を使用します。

可変リスト (`MutableList`)を作成するには、`mutableListOf()`関数を使用します。

リストを作成する際、Kotlin は格納される項目の型を推論することができます。型を明示的に宣言するには、リストの宣言の後に山括弧 `<>` で囲んだ型を追加します。

```
fun main() {
    //sampleStart
    // 読み取り専用リスト
    val readOnlyShapes = listOf("triangle", "square", "circle")
    println(readOnlyShapes)
    // [triangle, square, circle]

    // 明示的な型宣言を持つ可変リスト
    val shapes: MutableList<String> = mutableListOf("triangle", "square",
"circle")
    println(shapes)
    // [triangle, square, circle]
    //sampleEnd
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-lists-declaration"}

意図しない変更を防ぐために、変更可能なリストを `List` 型の別の新しい変数に代入することで、そのリストの読み取り専用ビューを作成することができます：

```
val shapes: MutableList<String> = mutableListOf("triangle", "square",
"circle")
```

```
val shapesLocked: List<String> = shapes
```

これは**キャスト**とも呼ばれます。

{style="tip"}

リストは順序付けられているため、リスト内の項目にアクセスするには、**インデックスアクセス演算子** `[]` を使用します：

```
fun main() {
//sampleStart
    val readOnlyShapes = listOf("triangle", "square", "circle")
    println("The first item in the list is: ${readOnlyShapes[0]}")
    // The first item in the list is: triangle
//sampleEnd
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-list-access"}

リストの先頭または末尾の要素を取得するには、それぞれ `.first()` および `.last()` 関数を使用します。

```
fun main() {
//sampleStart
    val readOnlyShapes = listOf("triangle", "square", "circle")
    println("The first item in the list is: ${readOnlyShapes.first()}")
    // The first item in the list is: triangle
//sampleEnd
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-list-first"}

`.first()` および `.last()` 関数は、**拡張関数**の例です。オブジェクトに対して拡張関数を呼び出すには、オブジェクト名の後にピリオド `.` を付けて関数名を記述します。

拡張関数については、**中級ツアー**で詳しく解説されています。とりあえずは、それらを呼び出す方法を知っておけば十分です。

{style="note"}

リスト内の要素数を取得するには、`.count()` 関数：を使用します。

```
fun main() {
//sampleStart
    val readOnlyShapes = listOf("triangle", "square", "circle")
    println("This list has ${readOnlyShapes.count()} items")
    // This list has 3 items
}
```

```
//sampleEnd  
}
```

```
{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-list-count"}
```

ある項目がリストに含まれているかどうかを確認するには、`in` 演算子を使用します：

```
fun main() {  
    //sampleStart  
    val readOnlyShapes = listOf("triangle", "square", "circle")  
    println("circle" in readOnlyShapes)  
    // true  
    //sampleEnd  
}
```

```
{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-list-in"}
```

変更可能なリストに項目を追加または削除するには、`.add()` および `.remove()` 関数をそれぞれ使用します：

```
fun main() {  
    //sampleStart  
    val shapes: MutableList<String> = mutableListOf("triangle", "square",  
    "circle")  
    // Add "pentagon" to the list  
    shapes.add("pentagon")  
    println(shapes)  
    // [triangle, square, circle, pentagon]  
  
    // Remove the first "pentagon" from the list  
    shapes.remove("pentagon")  
    println(shapes)  
    // [triangle, square, circle]  
    //sampleEnd  
}
```

```
{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-list-add-remove"}
```

Set

セット

リストは順序付けられており、重複する要素を含めることができますが、セットは順序がないものであり、一意な要素のみを格納します。

読み取り専用のセット (`Set`) を作成するには、`setOf()` 関数を使用します。

可変セット (`MutableSet`)を作成するには、`mutableSetOf()`関数を使用します。

セットを作成する際、Kotlin は格納される要素の型を推論できます。型を明示的に宣言するには、セットの宣言の後に山括弧 `<>` で囲んだ型を追加します：

```
fun main() {
    //sampleStart
    // 読み取り専用セット
    val readOnlyFruit = setOf("apple", "banana", "cherry", "cherry")
    // 明示的な型宣言を持つ可変セット
    val fruit: MutableSet<String> = mutableSetOf("apple", "banana", "cherry",
    "cherry")

    println(readOnlyFruit)
    // [apple, banana, cherry]
    //sampleEnd
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-sets-declaration"}

前の例からもわかるように、セットには重複のない要素しか含まれないため、重複している「cherry」という項目は除外されます。

To prevent unwanted modifications, you can create a read-only view of a mutable set by assigning it to a `Set`:

```
val fruit: MutableSet<String> = mutableSetOf("apple", "banana",
"cherry", "cherry")
val fruitLocked: Set<String> = fruit
```

{style="tip"}

集合は順序付けられていないため、特定のインデックスにある要素にアクセスすることはできません。

{style="note"}

集合に含まれる要素の数を取得するには、`.count()` 関数を使用します：

```
fun main() {
    //sampleStart
    val readOnlyFruit = setOf("apple", "banana", "cherry", "cherry")
    println("This set has ${readOnlyFruit.count()} items")
    // This set has 3 items
    //sampleEnd
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-set-count"}

ある要素が集合に含まれているかどうかを確認するには、**in 演算子** を使用します：

```
fun main() {
//sampleStart
    val readOnlyFruit = setOf("apple", "banana", "cherry", "cherry")
    println("banana" in readOnlyFruit)
    // true
//sampleEnd
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-set-in"}

可変セットに要素を追加または削除するには、**[.add()]**

(<https://kotlinlang.org/api/latest/jvm/stdlib/kotlin.collections/-mutable-set/add.html>) および **.remove()** 関数をそれぞれ使用します：

```
fun main() {
//sampleStart
    val fruit: MutableSet<String> = mutableSetOf("apple", "banana", "cherry",
"cherry")
    fruit.add("dragonfruit")    // セットに "dragonfruit" を追加
    println(fruit)             // [apple, banana, cherry, dragonfruit]

    fruit.remove("dragonfruit") // セットから "dragonfruit" を削除
    println(fruit)             // [apple, banana, cherry]
//sampleEnd
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-set-add-remove"}

Map

マップ

マップは、項目をキーと値のペアとして格納します。キーを参照することで、その値にアクセスできます。マップは、レストランのメニューのようなものだと考えてください。食べたい食べ物（キー）を見つけることで、その価格（値）を知ることができます。マップは、リストのように番号付きのインデックスを使わずに値を調べたい場合に役立ちます。

- マップ内のキーはすべて一意である必要があります。そうすることで、Kotlinはどの値を取得したいのかを判別できるようになります。
- マップには重複する値を格納することができます。

{style="note"}

読み取り専用のマップ (**Map**) を作成するには、**mapOf()** 関数を使用します。

可変マップ (**MutableMap**) を作成するには、**mutableMapOf()** 関数を使用します。

マップを作成する際、Kotlin は格納される項目の型を推論することができます。型を明示的に宣言するには、マップの宣言の後に、角括弧 `<>` で囲んだキーと値の型を追加します。例：`MutableMap<String, Int>`。キーの型は `String` で、値の型は `Int` です。

マップを作成する最も簡単な方法は、各キーとその関連する値の間に `to` を使用することです：

```
fun main() {
//sampleStart
    // Read-only map
    val readOnlyJuiceMenu = mapOf("apple" to 100, "kiwi" to 190, "orange" to 100)
    println(readOnlyJuiceMenu)
    // {apple=100, kiwi=190, orange=100}

    // Mutable map with explicit type declaration
    val juiceMenu: MutableMap<String, Int> = mutableMapOf("apple" to 100, "kiwi"
to 190, "orange" to 100)
    println(juiceMenu)
    // {apple=100, kiwi=190, orange=100}
//sampleEnd
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-maps-declaration"}

T意図しない変更を防ぐために、変更可能なマップを `Map` に代入することで、そのマップの読み取り専用ビューを作成することができます：

```
val juiceMenu: MutableMap<String, Int> = mutableMapOf("apple" to 100,
"kiwi" to 190, "orange" to 100)
val juiceMenuLocked: Map<String, Int> = juiceMenu
```

{style="tip"}

マップ内の値にアクセスするには、キーとともに `インデックスアクセス演算子` `[]` を使用します：

```
fun main() {
//sampleStart
    // Read-only map
    val readOnlyJuiceMenu = mapOf("apple" to 100, "kiwi" to 190, "orange" to 100)
    println("The value of apple juice is: ${readOnlyJuiceMenu["apple"]}")
    // The value of apple juice is: 100
//sampleEnd
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-map-access"}

マップに存在しないキーを持つキーと値のペアにアクセスしようとすると、`null` が表示されます：

```
fun main() {
//sampleStart
    // Read-only map
    val readOnlyJuiceMenu = mapOf("apple" to 100, "kiwi" to 190, "orange" to 100)
    println("The value of pineapple juice is:
    ${readOnlyJuiceMenu["pineapple"]}")
    // The value of pineapple juice is: null
//sampleEnd
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-map-no-key" validate="false"}

このツアーでは、[Null Safety](#)の章の後半で、null値について解説しています。

{style="note"}

また、[インデックスアクセス演算子](#) `[]` を使用して、可変マップに項目を追加することもできます：

```
fun main() {
//sampleStart
    val juiceMenu: MutableMap<String, Int> = mutableMapOf("apple" to 100, "kiwi"
to 190, "orange" to 100)
    juiceMenu["coconut"] = 150 // マップに、キー「coconut」、値150を追加する
    println(juiceMenu)
    // {apple=100, kiwi=190, orange=100, coconut=150}
//sampleEnd
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-map-add-item"}

変更可能なマップから項目を削除するには、`.remove()`関数を使用します。

```
fun main() {
//sampleStart
    val juiceMenu: MutableMap<String, Int> = mutableMapOf("apple" to 100, "kiwi"
to 190, "orange" to 100)
    juiceMenu.remove("orange") // マップから「orange」というキーを削除する
    println(juiceMenu)
    // {apple=100, kiwi=190}
//sampleEnd
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-map-put-remove"}

マップ内の要素の数を取得するには、`.count()` 関数を使用します：

```
fun main() {
//sampleStart
    // Read-only map
    val readOnlyJuiceMenu = mapOf("apple" to 100, "kiwi" to 190, "orange" to 100)
    println("This map has ${readOnlyJuiceMenu.count()} key-value pairs")
    // This map has 3 key-value pairs
//sampleEnd
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-map-count"}

特定のキーがマップにすでに含まれているかどうかを確認するには、`.containsKey()` 関数を使用します：

```
fun main() {
//sampleStart
    val readOnlyJuiceMenu = mapOf("apple" to 100, "kiwi" to 190, "orange" to 100)
    println(readOnlyJuiceMenu.containsKey("kiwi"))
    // true
//sampleEnd
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-map-contains-keys"}

マップのキーまたは値のコレクションを取得するには、`keys` および `values` プロパティをそれぞれ使用します：

```
fun main() {
//sampleStart
    val readOnlyJuiceMenu = mapOf("apple" to 100, "kiwi" to 190, "orange" to 100)
    println(readOnlyJuiceMenu.keys)
    // [apple, kiwi, orange]
    println(readOnlyJuiceMenu.values)
    // [100, 190, 100]
//sampleEnd
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-map-keys-values"}

`keys` および `values` は、オブジェクトの **プロパティ** の例です。オブジェクトのプロパティにアクセスするには、オブジェクト名の後にピリオド `.` を付けてプロパティ名を記述します。

プロパティについては、「[クラス](#)」の章でさらに詳しく説明しています。ツアーのこの段階では、それらにアクセスする方法さえ知っていれば十分です。

{style="note"}

マップにキーまたは値が含まれているかどうかを確認するには、`in` **演算子** を使用します：

```
fun main() {
    //sampleStart
    val readOnlyJuiceMenu = mapOf("apple" to 100, "kiwi" to 190, "orange" to 100)
    // キーに「orange」が含まれているかどうか
    println("orange" in readOnlyJuiceMenu.keys)    // true

    // あるいは、keys プロパティを使用する必要はありません
    println("orange" in readOnlyJuiceMenu)        // true

    // 値に200が含まれているかどうか
    println(200 in readOnlyJuiceMenu.values)      // false
    //sampleEnd
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-map-in"}

コレクションの活用方法に関する詳細については、[コレクション](#)をご覧ください。

基本的な型やコレクションの管理方法について理解できたところで、次はプログラムで利用できる[制御フロー](#)について見ていきましょう。

Practice {completion-point="true"}

演習 {completion-point="true"}

Exercise 1 {initial-collapse-state="collapsed" collapsible="true"}

課題 1 {initial-collapse-state="collapsed" collapsible="true"}

「緑」の数字のリストと「赤」の数字のリストがあります。合計でいくつの数字があるかを表示するように、コードを完成させてください。

|---|---|

```
fun main() {
    val greenNumbers = listOf(1, 4, 23)
    val redNumbers = listOf(17, 2)
    // ここにコードを入力してください
}
```

{validate="false" kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-collections-exercise-1"}

|---|---|

```
fun main() {
    val greenNumbers = listOf(1, 4, 23)
    val redNumbers = listOf(17, 2)
```

```
    val totalCount = greenNumbers.count() + redNumbers.count()
    println(totalCount)
}
```

{initial-collapse-state="collapsed" collapsible="true" collapsed-title="Example solution" id="kotlin-tour-collections-solution-1"}

Exercise 2 {initial-collapse-state="collapsed" collapsible="true"}

課題 2 {initial-collapse-state="collapsed" collapsible="true"}

サーバーがサポートしているプロトコルのセット(例: "HTTP", "HTTPS", "FTP")があります。ユーザーが特定のプロトコルの使用をリクエストしました。リクエストされたプロトコルがサポートされているかどうかを確認するプログラムを完成させてください (`isSupported` はブール値でなければなりません)。

|---|---|

```
fun main() {
    val SUPPORTED = setOf("HTTP", "HTTPS", "FTP")
    val requested = "smtp"
    val isSupported = // Write your code here
    println("Support for $requested: $isSupported")
}
```

{validate="false" kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-collections-exercise-2"}

指定されたプロトコルが大文字であることを必ず確認してください。この作業には、`.uppercase()` 関数を利用できます。

|---|---|

```
fun main() {
    val SUPPORTED = setOf("HTTP", "HTTPS", "FTP")
    val requested = "smtp"
    val isSupported = requested.uppercase() in SUPPORTED
    println("Support for $requested: $isSupported")
}
```

{initial-collapse-state="collapsed" collapsible="true" collapsed-title="Example solution" id="kotlin-tour-collections-solution-2"}

Exercise 3 {initial-collapse-state="collapsed" collapsible="true"}

課題 3 {initial-collapse-state="collapsed" collapsible="true"}

1 から 3 までの整数と、その整数を表す英語の綴りを結びつける対応関係を定義してください。この対応関係を用いて、与えられた 番号に対応する綴りを表示してください。

|---|---|

```
fun main() {  
    val number2word = // ここにコードを入力してください  
    val n = 2  
    println("$n is spelled as '${<Write your code here >}''")  
}
```

{validate="false" kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-collections-exercise-3"}

|---|---|

```
fun main() {  
    val number2word = mapOf(1 to "one", 2 to "two", 3 to "three")  
    val n = 2  
    println("$n is spelt as '${number2word[n]}''")  
}
```

{initial-collapse-state="collapsed" collapsible="true" collapsed-title="Example solution" id="kotlin-tour-collections-solution-3"}

- [Previous step](#)
- [Next step](#)